

04.06.25

What is the goal now?

The goal now is to push my encodings to the state where they can handle Kep's environments. But first, let's jump back to what we've had most recently just to recap:

a. I've managed to run some really complex environments like 10x10 with 3 trains. Maybe, I'll jump back to them now.

b. I also tried running Kep's envs (huge but with just 2 trains), but it was less successful.

c. Maybe it will be a good idea to have a well-developed intermediate iteration between 10x10 and 30x30.

Good, on 10x10 with as many as 4 trains it finds 20 stable models in (Solving: 9.13s 1st Model: 1.51s Unsat: 0.24s).

Now, there is no point in adding more trains to this. What's then? I would play around with 15x15 or 20x20.

It would be inconvenient to implement a new 15 by 15 now since I'm on the train, so let's deal with 20x20, since it's already encoded as an empty environment (however, it's almost untested, so I must remember that there might be mistakes.

A small thing, but let's change the file format for the visual representation of 20x20 for consistency: done.

Let's run the simplest single train on it. It includes one choice point perhaps. Yes, it seems that only one. Let's check if the start and the end are encoded.

I've also tried running my 10x10 encoding with 4 trains but without giving an extra leeway for one of them (3rd) and it proved to be very fast: just 1.3 seconds to solve the environment. What I should now before I'm able to experiment with 20x20 is converting it to Kep's coordinates. I have a special python tool for this. Let's use it to convert the 20x20 into the new system (I can easily convert start and finish using this tool as well by masking them as 'cell' atoms first): done.

Now let's try to run in. Expectation: it fails due to a naive bug somewhere. Let's blindly run it for now. I was right, it fails. Let's debug.

H1: time is not enough - NO: increasing time doesn't help anything.

H2: the start direction is wrong; no, the start direction is correct.

H3: there is a missing track type description

Debugging (H3 is the priority hypothesis): Let's backtrack by starting with the shortest path and then increasing the length.

Discovery: there is a **brutal mistake** in the environment representation. Fixing: this probably can be an edge case for my coordinate converter. Since it's the last environment I need to convert, let's do it stupidly by hand instead of fixing the helper.

05.06.25

Okay, what's the next thing to do?

- a. Finish the conversion of the 20x20 environment. Done. Let's test if it runs now (there's still a high likelihood of a mistake somewhere; checking additionally: yes, the conversion must be correct - the numbers now are sorted in the descending order).
- b. Run 1 train on it: **fails**. Why? Analysing: the start coordinate is **completely wrong** - it's actually out of the bounds of the environment (conclusion: my converter falters when dealing with 20x20, but it's not a big deal since I'm done using it).

Now: assign a new start and new end. The end was correct. **Fixed**. Let's try running it now: **fails**. Let's backtrack.

DISCOVERY: my encoding permits waiting in one cell for as long as necessary!

```
transition(0,0,155,east,wait,155,east) transition(0,1,155,east,wait,155,east)
transition(0,2,155,east,wait,155,east) transition(0,3,155,east,wait,155,east)
transition(0,4,155,east,wait,155,east) transition(0,5,155,east,wait,155,east)
transition(0,6,155,east,wait,155,east) transition(0,7,155,east,move_forward,156,east)
```

There is an error in the environment description. Or it is correct, but 1097 track type is not described. Checking: it is described. Then: is it described correctly? Debugging: observation: it all starts to work once I replace track of type 1097 in the position (17, _) with 1025 (simple east-west deterministic piece).

IDEA: "path" atoms can be packed further into "nodes" before applying a choice rule to make the model faster.

In reading 'path' atoms, I must not forget that now the coordinate system is upside down.

Still can't catch the bug. After replacing, the encoding has worked.

OBSERVATION: something's off with 1097 - but I don't see what.
Fails on 2065 corner turn. Debugging: fails **disregarding the type**.

Idea: stop debugging my environment for a while and try to use Clingonia to generate one instead. Let's go:

Exploring Clingonia

-  `asp` is where YOU can put your `lp` files (working `lp` files included).
-  `code` contains Python source codes.
-  `data` contains images, saved parameters, helper texts, and runtime information.
-  `env` contains environment `lp` encodings.
-  `main.py` is the primary file for launching the program.
-  `README.md` is what you are reading right now.

I need to run '[main.py](#)' then. The repository is already cloned locally. Let's try running it; but first let's create the virtual environment and install the required packages:

```
clingo , imageio , pillow , numpy , pandas , matplotlib .
```

Clingo +
Imageio +
Pillow +
Numpy +
Pandas +
Matplotlib +

Done. Now let's try to run [main.py](#): **fails** => flatland package is missing. Installing: done.
Running the main file again: **fails**. Am I doing the right thing?

```
python main.py
```



This will launch the program.

Yes. Yes it fails. The error message:

```

Traceback (most recent call last):
  File "C:\Users\roman\Clingonia\main.py", line 79, in <module>
    main()
    ~~~~~^
  File "C:\Users\roman\Clingonia\main.py", line 74, in main
    initial_render_authorization() # Test that rendering pipeline is functional
    ~~~~~^
  File "C:\Users\roman\Clingonia\main.py", line 44, in initial_render_authorization
    initial_render_test()
    ~~~~~^
  File "C:\Users\roman\Clingonia\code\build_png.py", line 279, in initial_render_test
    env.reset()
    ~~~~~^
  File "C:\Users\roman\Clingonia\venv\Lib\site-packages\flatland\envs\rail_env.py", line 328, in reset
    self.agents = EnvAgent.from_line(line)
    ~~~~~^
  File "C:\Users\roman\Clingonia\venv\Lib\site-packages\flatland\envs\agent_utils.py", line 180, in from_line
    agent = EnvAgent(initial_position=line.agent_positions[i_agent][0],
    ~~~~~^
TypeError: 'NoneType' object is not subscriptable

```

Let's try to debug quickly with an LLM. If it doesn't help, proceed to implementing our own environments / testing on Kep's.

LLM: *"Error Location: The error is occurring in the from_line function in agent_utils.py at line 180. The line in question is trying to access an index of initial_position, which is None."*

But there isn't even a file with this name!

Let's write to the forum: **Done**. Maybe, we'll be helped. No, not more debugging. Let's build a 15x15 env?

Let's write a python helper to create an empty environment of arbitrary size: oh, what am I doing? I've done so much that I forgot parts of it. I already have this canvas generator:

Env_generator.py

Let's rename it for more explicitness into 'canvas_generator.py': **Done**.

Generating a 15x15 canvas: **Done**.

Filling out the tracks. **Done**. (it's incredibly taxing on the brain - I need to have a tool for this unless Clingonia works).

Let's run first 1, then a few trains on it. Two ways now:

- a. Run a bit on 15x15, analyse.
- b. Experiment with handdrawings converter tool.

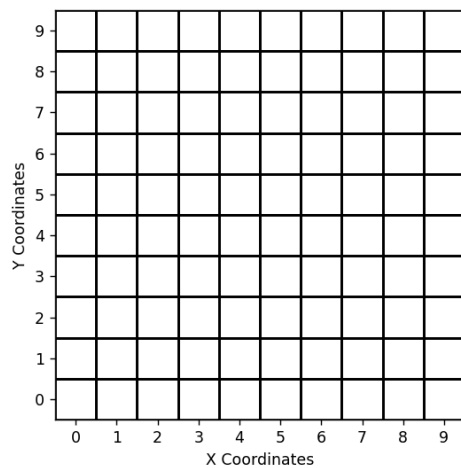
Test	Result	Time
1 (1 train)	Success	(Solving: 0.00s 1st Model: 0.00s Unsat: 0.00s)
2 (1 train)	Failure => Success	(Solving: 0.00s 1st Model: 0.00s Unsat: 0.00s)
3 (1 train)	Failure => Success	(Solving: 0.00s 1st Model: 0.00s Unsat: 0.00s)
4 (2 trains)	Failure => Success	(Solving: 0.01s 1st Model: 0.01s Unsat: 0.00s)
5 (3 trains)	Failure	(Solving: 0.00s 1st Model: 0.00s Unsat: 0.00s)

Debugging test 2 by shortening: no bug found, but it works (marvellous!). Same for the next test. Perhaps I initially miscalculate time.

INTUITION: there could be a problem of trains standing in the arrival position and waiting, but the trains “disappear” upon arrival (**I should check this behaviour in a very simple environment**).

OBSERVATION: the current encoding doesn't prohibit two (or more) trains to start from the same position at the same moment of time in different directions (which is **good**).

Let's play around with Kep's smallest environments again. Let's only run one train first. It fails. And so far I don't see how to debug it. One of the ways is to run a Python helper that takes the cell description of the environment and outputs it on a clear grid.



Managed to obtain this so far: it takes in the coordinates and outputs the grid. Next step is to locate images by numbers, which must be very easy actually.

06.05.25

Now, I need to make the mapping between the TYPE variable in every cell and the image, putting the image into the respective cell. I also need to add 0 cell type. **Done.**

Now to the mapping. First, let's just plot the type in each cell - without fetching the respective tile. What datatype should it be? Can a tuple be a key in the dictionary?

LLM: “Yes, in Python, a tuple can be used as a key in a dictionary because tuples are immutable. This means that once a tuple is created, its contents cannot be changed, making it a suitable candidate for a dictionary key.”

This is **done**. Let's think how we can extract the type from here:

```
['cell((0, 0), 32800)', 'cell((0, 1), 0)', 'cell((0, 2), 0)', 'cell((1, 0), 32800)', 'cell((1, 1), 0)', 'cell((1, 2), 0)', 'cell((2, 0), 32800)', 'cell((2, 1), 0)', 'cell((2, 2), 0)']
```

The TYPE value always starts at the same position, but **ends at a different one**, so the easiest way is to: take everything from a certain position, remove unnecessary characters. The TYPE variable always starts from 14th position. **Done**. Let's test on bigger instances: **works**.

Let's now tile the grid with the type codes: **done**.

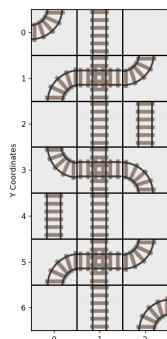
2	32800	0	0
1	32800	0	0
0	32800	0	0
	0	1	2

Good. Now let's test on bigger instances. I need to make the font in the tiles smaller. **Success**.

Now the final step: let's make the actual tiling: **Done**.

Let's test on different environments.

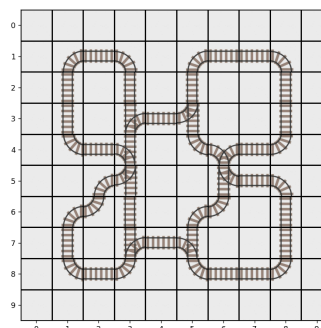
BUG: the coordinates are not in Kep's 'system'.
Fixing: **done**.



Still broken - every turn (or every segment in general) is simply upside down. Let's simply turn every image upside down instead of fiddling with the code. But first of course copy the whole folder for safety.

Fixed. Testing all environments: **fails**. Debugging: **done**.

Sample large output:



IDEA: hand drawer (if I implement one) + visualizer is a perfect toolkit for development.

Now I can debug my encoding conveniently by placing trains in different positions.

Debugging my encoding on Kep's large environments:

test	start	end	result	time
1	(0, (24,11), 0, north)	(0, (15,8), 12)	success	(Solving: 0.00s 1st Model: 0.00s Unsat: 0.00s)
2	(0, (24, 7), 0, south)	(0, (5,11), 42)	failure H1: there is an error in my path description, most likely in the type with the choice. NO. Reason for bug: no waiting is allowed, for some reason. Something in my encoding prohibits waiting now. Strange behaviour: switching off the constraint on ghost movement led to increasing computation time dramatically.	
3				
4				
5				
6				
7				
8				
9				
10				

INTUITION: a debugging tool is possible that iteratively back-tracks the path to report where the encoding fails.

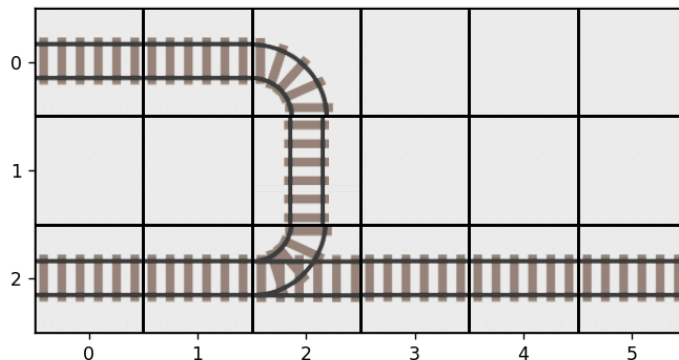
OPTIMIZATION: only allowing to wait before the junction makes sense to reduce grounding.

Let's try to develop such a constraint:

Constraint Idea	ASP phrasing	Code
Waiting is only possible before a junction.	There must be no answer set where there is a 'wait' path atom in the position other than that before the junction.	

Instead of the constraint, I can try to simply modify the choice rule in such a way that 'wait' containing atoms are only created before junctions:

If there is a junction path, then the wait path atom must be created for the previous position. The best is to find examples:



So, waiting must only be allowed at (2, 1) and at (1, 2).

Let's display the 'path' atoms:

```

path(north,8,move_forward,north,2)
path(south,8,move_forward,south,14)
path(east,12,move_forward,east,13)
path(east,13,move_forward,east,14)
path(east,15,move_forward,east,16)
path(east,16,move_forward,east,17)
path(east,0,move_forward,east,1)
path(east,1,move_forward,east,2) path(west,13,move_forward,west,12)
path(west,15,move_forward,west,14) path(west,16,move_forward,west,15)
path(west,17,move_forward,west,16) path(west,1,move_forward,west,0)
path(east,2,move_forward,south,8) path(north,2,move_forward,west,1)
path(east,14,move_forward,east,15) path(east,14,turn_left,north,8)
path(south,14,move_forward,west,13) path(west,14,move_forward,west,13)
path(west,14,wait,west,14) path(south,14,wait,south,14) path(east,14,wait,east,14)
path(north,2,wait,north,2) path(east,2,wait,east,2) path(west,1,wait,west,1)
path(west,17,wait,west,17) path(west,16,wait,west,16) path(west,15,wait,west,15)
path(west,13,wait,west,13) path(east,1,wait,east,1) path(east,0,wait,east,0)
path(east,16,wait,east,16) path(east,15,wait,east,15) path(east,13,wait,east,13)
path(east,12,wait,east,12) path(south,8,wait,south,8) path(north,8,wait,north,8)

```

Let's highlight the junction atom (only 1 here). Naively: if there is a path whose outgoing position is 14, then that is the only location where waiting can happen:

```

is_junction(IN_POS) :- path(IN_DIR, IN_POS, ACTION_1, _, _),
path(IN_DIR, IN_POS, ACTION_2, _, _).

```

07.06.25

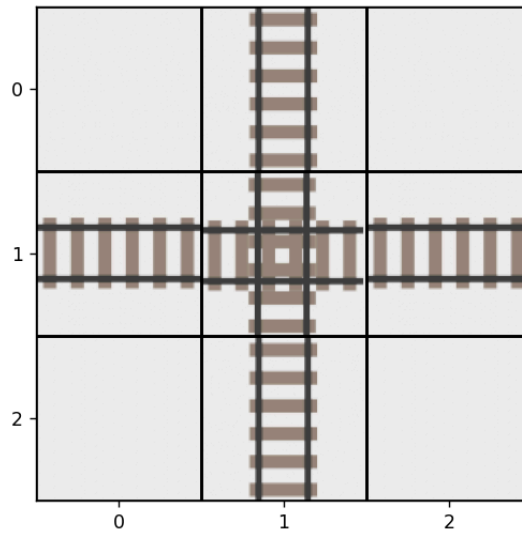
Let's keep debugging:

- a. What I'm trying to achieve now is to make the trains wait only before junctions.

Let's explore this in the simplest environments only. The absolute simplest switching environment is the 'diamond'. However, the problem with it might be that waiting there is *only possible* before a switch. Let's experiment this way:

Converting 'diamond for two' into the new coordinates: **done**.

Trying to run with the new waiting logic: **fails**. Fixing: the normal rule generating the 'wait' path atoms was commented out: **fixed**. Let's explore the output:



```
action(1,0,move_forward) action(1,1,move_forward) action(2,0,wait) action(2,1,move_forward)
action(2,2,wait) action(2,3,move_forward)
```

Answer: 2

```
action(1,0,move_forward) action(1,1,move_forward) action(2,0,wait) action(2,1,wait)
action(2,2,move_forward) action(2,3,move_forward)
```

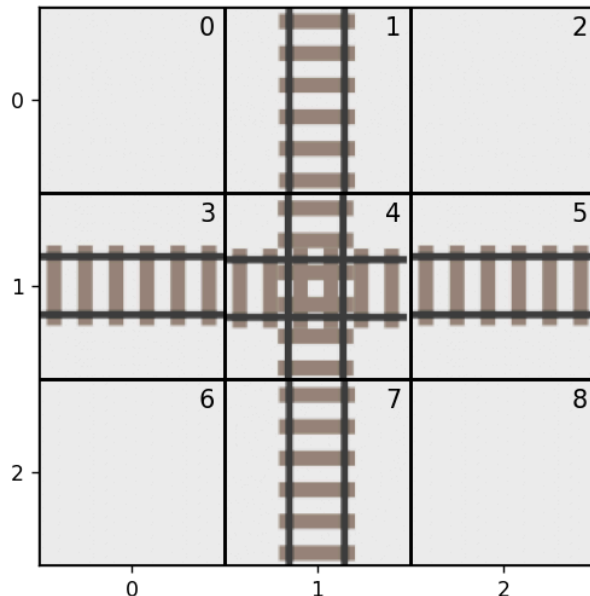
The output looks correct. Let's try to shorten the leeway for the second train by 1 and see the behaviour:

```
action(1,0,move_forward) action(1,1,move_forward) action(2,0,wait) action(2,1,move_forward)
action(2,2,move_forward)
```

Only one answer - as expected. Waiting only occurs in the cell before the junction. Let's increase the leeway for the second train further and see the behaviour:

Good, increasing the leeway arbitrarily doesn't break the program. Let's explore other underlying atoms beyond just 'action': looking at 'transition' will help us see if the train 'violates' the (intention of the) normal rule and stands right at the junction:

For readability, it would be great to add the 1d positional encoding to the python helper. **Done.**



Let's now explore the transitions:

```
transition(1,0,3,east,move_forward,4,east)
transition(1,1,4,east,move_forward,5,east)
transition(2,0,1,south,wait,1,south)
transition(2,1,1,south,move_forward,4,south)
transition(2,2,4,south,wait,4,south)
transition(2,3,4,south,move_forward,7,south)
```

Here, waiting occurs right at the junction. How is it possible?

```
transition(1,0,3,east,move_forward,4,east)
transition(1,1,4,east,move_forward,5,east)
transition(2,0,1,south,wait,1,south)
transition(2,1,1,south,wait,1,south)
transition(2,2,1,south,move_forward,4,south)
transition(2,3,4,south,move_forward,7,south)
```

Here, there's no waiting at the junction. Let's analyse the normal rule for waiting creation:
Underwood:

```
path(IN_DIR, POSITION, wait, IN_DIR, POSITION):- path(IN_DIR, POSITION, _,
_, _).
```

I have doubled the normal rule. Switching off and trying to run: **fails**. Let's explore the path atoms (INTUITION: 'wait' atoms are simply not created):

```
path(north,7,move_forward,north,4) path(south,1,move_forward,south,4) path(east,3,move_forward,east,4)
path(west,5,move_forward,west,4) path(north,4,move_forward,north,1) path(south,4,move_forward,south,7)
path(east,4,move_forward,east,5) path(west,4,move_forward,west,3)
```

Intuition **confirmed**. Solution? Problem: junction is also **not identified**. Let's fix this: it feels like the easiest way to spot junctions right from the input - by index. Tested on many environments up to the biggest. **Works**. We go on: the easiest might be to derive the waiting position from path atoms. The normal rule must sound like this:

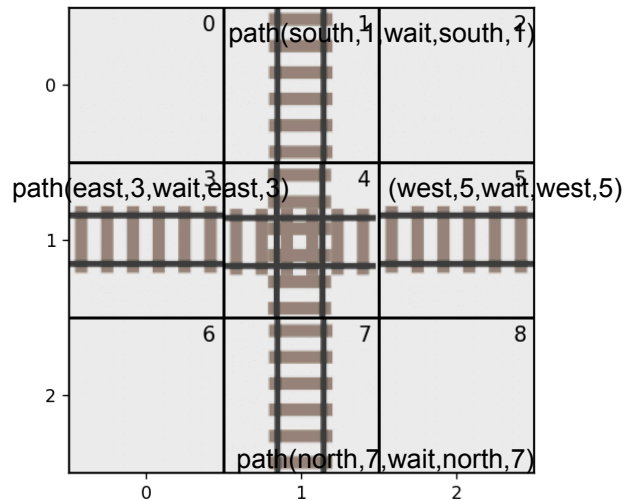
Rule idea	ASP phrasing	Code
If there is a path atom leading	If there is a path atom that is	path(IN_DIR, IN_POS, wait, IN_DIR,

to a junction, then there must be created a path atom created for this position where the train can wait.	junction and a path atom that leads to the junction, create a path atom with 'wait' with the same position / direction as the one that leads to the junction.	IN_POS) :- path(IN_DIR, IN_POS, _, OUT_POS, OUT_DIR), path(OUT_DIR, OUT_POS, _, _, _), is_junction(OUT_POS).
---	---	--

Let's check the output:

```
path(north,7,move_forward,north,4) path(south,1,move_forward,south,4)
path(east,3,move_forward,east,4) path(west,5,move_forward,west,4) path(north,4,move_forward,north,1)
path(south,4,move_forward,south,7) path(east,4,move_forward,east,5)
path(west,4,move_forward,west,3) path(north,7,wait,north,7) path(south,1,wait,south,1)
path(east,3,wait,east,3) path(west,5,wait,west,5)
```

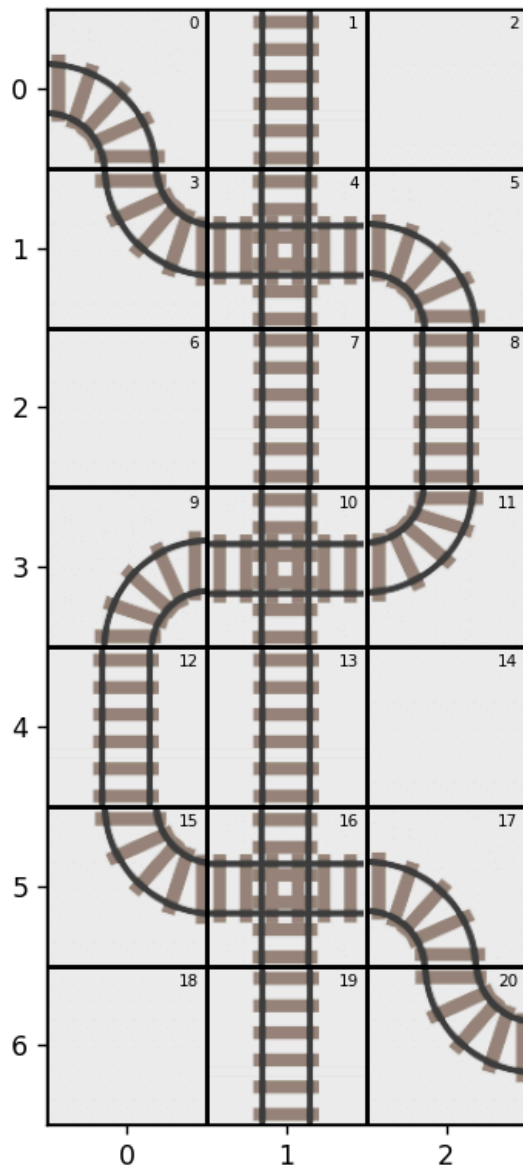
Let's see if the suggested waiting positions are correct: they must be only before the junction (ends of the cross) and never in the junction itself:



Perfect: waiting only occurs before junctions and the (directions of waiting) are correct. Let's scale this to bigger environments. Before, just increase leeway for the second train: works perfectly - the train can wait there for as long as I want (leeway of 30 moments below):

```
transition(1,0,3,east,move_forward,4,east) transition(1,1,4,east,move_forward,5,east) transition(2,0,1,south,wait,1,south)
transition(2,1,1,south,wait,1,south) transition(2,2,1,south,wait,1,south) transition(2,3,1,south,wait,1,south)
transition(2,4,1,south,wait,1,south) transition(2,5,1,south,wait,1,south) transition(2,6,1,south,wait,1,south)
transition(2,7,1,south,wait,1,south) transition(2,8,1,south,wait,1,south) transition(2,9,1,south,wait,1,south)
transition(2,10,1,south,wait,1,south) transition(2,11,1,south,wait,1,south) transition(2,12,1,south,wait,1,south)
transition(2,13,1,south,wait,1,south) transition(2,14,1,south,wait,1,south) transition(2,15,1,south,wait,1,south)
transition(2,16,1,south,wait,1,south) transition(2,17,1,south,wait,1,south) transition(2,18,1,south,wait,1,south)
transition(2,19,1,south,wait,1,south) transition(2,20,1,south,wait,1,south) transition(2,21,1,south,wait,1,south)
transition(2,22,1,south,wait,1,south) transition(2,23,1,south,wait,1,south) transition(2,24,1,south,wait,1,south)
transition(2,25,1,south,wait,1,south) transition(2,26,1,south,wait,1,south) transition(2,27,1,south,wait,1,south)
transition(2,28,1,south,move_forward,4,south) transition(2,29,4,south,move_forward,7,south)
```

The big one still fails. Let's backtrack to smaller ones: running Caduceus:



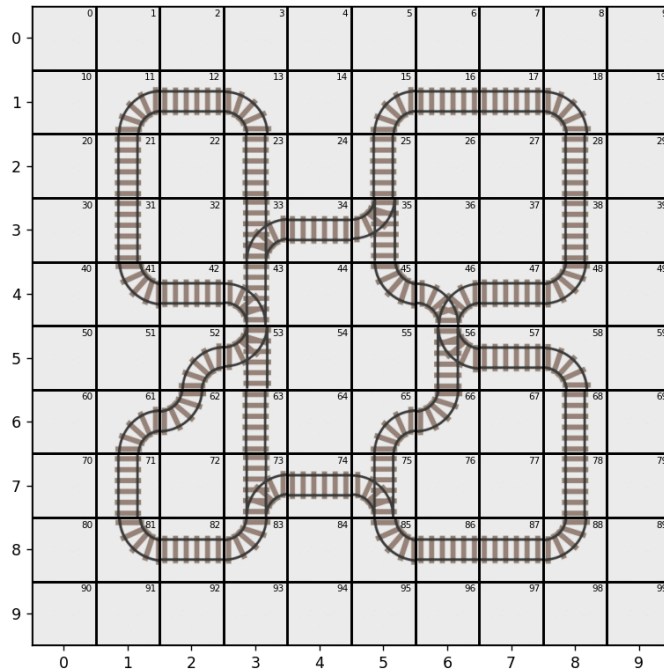
Solves in no time. Let's add waiting: **works**.

Strange bug: can't make the second train start later.

Understanding: waiting is not allowed for the second train at its starting position.

Solution (not implemented now): allowing the trains to wait at the starting points. No, I've still implemented it now. Testing: **works**.

Now, let's play around with 10 x 10 again:



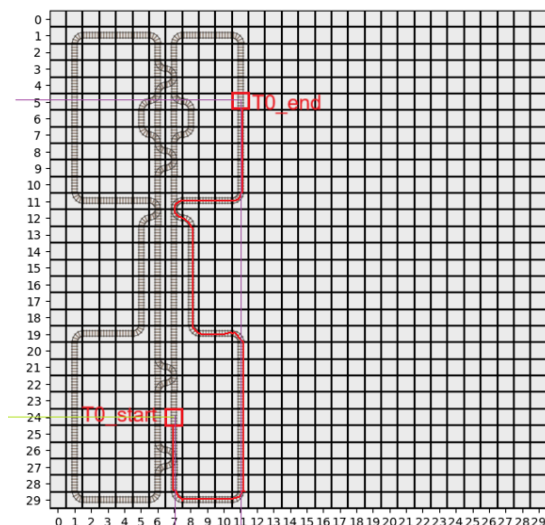
Running 4 trains on it. Two trains are conflicting. Perhaps not due to the encoding - they are just obviously crashing there. I need to make one of them wait for the second to arrive and 'disappear'. Let's set the right time: **fixed**. It was a purely 'domain' issue - unrelated to any problems with the encoding and even more - **confirming its correctness and functionality**.

All 4 trains also run well:

(Solving: 0.45s 1st Model: 0.06s Unsat: 0.01s)

Let's run Kep's environment again:

For now, just 1 train again:



Works. But hundreds of answers. Too much time? Wrong start and wrong end.

Pretty slow with 1 train. Would adding 1 train more radically slow down the program?

But first, let's run exactly Kep's train - one of them:

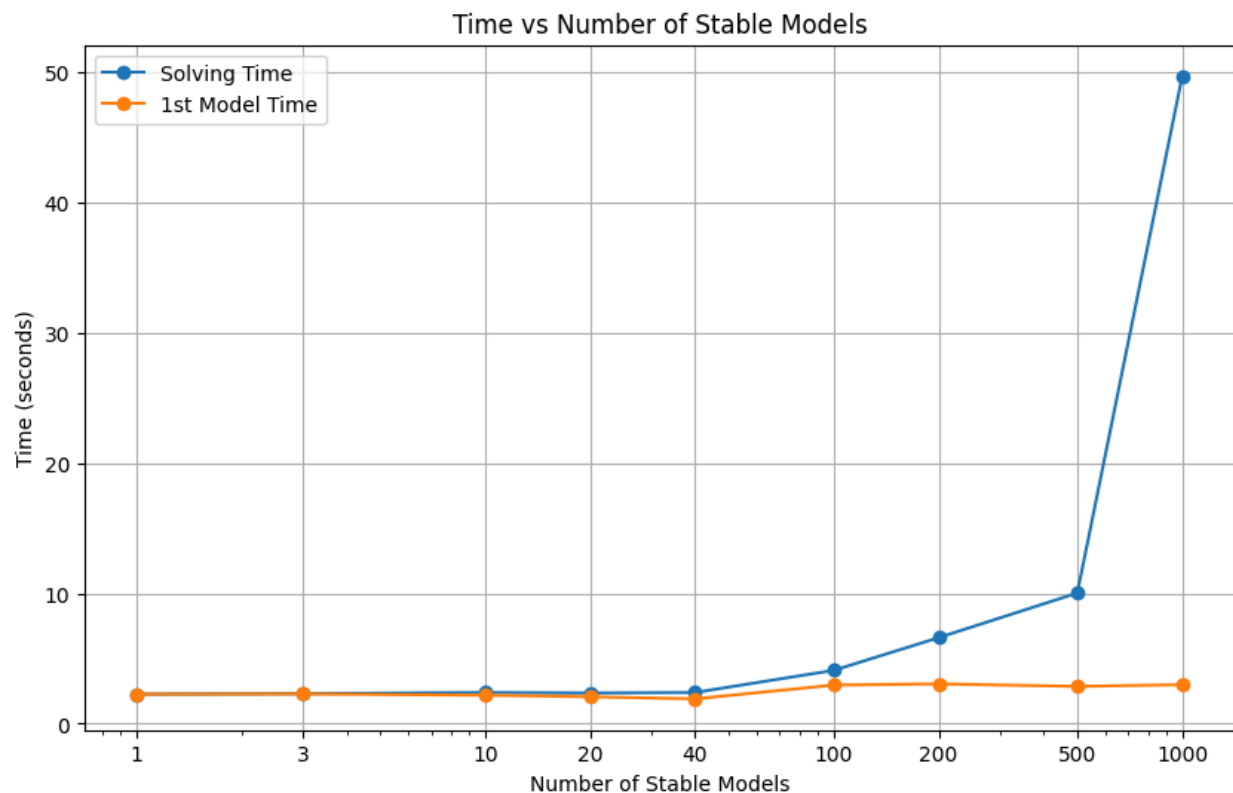
Runs *okay*.

OBSERVATION: setting up the output to be 'clingo 1' makes the program run fast - maybe even for bigger instances. The goal is to find a solution - the answer *is* a solution. **Success** with the 2nd train.

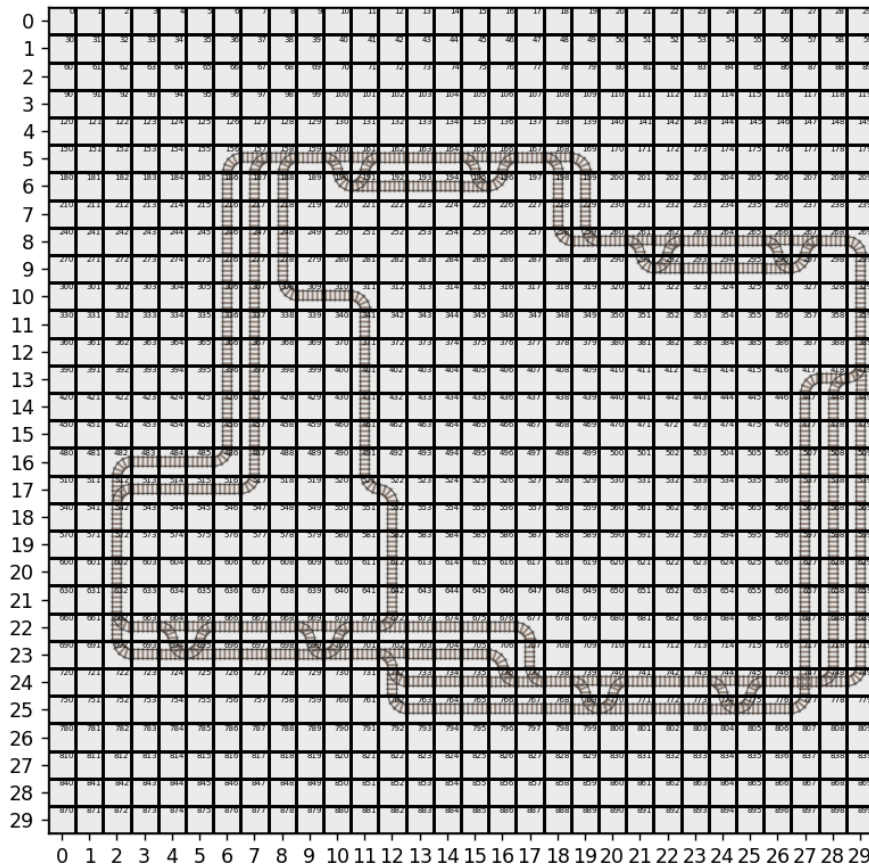
For **2 trains** it finds

1 stable model: (Solving: 2.27s 1st Model: 2.27s Unsat: 0.00s)
3 stable models: (Solving: 2.31s 1st Model: 2.29s Unsat: 0.00s)
10 stable models: (Solving: 2.40s 1st Model: 2.19s Unsat: 0.00s)
20 stable models: (Solving: 2.36s 1st Model: 2.07s Unsat: 0.00s)
40 stable models: (Solving: 2.40s 1st Model: 1.91s Unsat: 0.00s)
100 stable models: (Solving: 4.10s 1st Model: 2.97s Unsat: 0.00s)
200 stable models: (Solving: 6.61s 1st Model: 3.06s Unsat: 0.00s)
500 stable models: (Solving: 10.04s 1st Model: 2.87s Unsat: 0.00s)
1000 stable models: (Solving: 49.73s 1st Model: 3.00s Unsat: 0.00s)

Let's plot this relationship:



The time for solving 1000 stable models is still very much acceptable. As far as I understand the solver, '*harnessing*' these 1000 models with '*minimize*' won't be a problem. For now, let's proceed to the second Kep's environment. Let's rush in, running both trains at once. But first still visualise it to get the feel of what it is (and what implications it has for the solver):



Looks like it covers a bigger area with paths - which should imply more work for the solver.

But let's check it.
Running: **freezes**.
'Solving' displays.

No answer in:

(Solving: 552.13s 1st
Model: 0.00s Unsat:
0.00s)

Let's try the 3rd one and it will be enough for today: surprisingly, the 3rd one is faster solvable, but train 1 is unsatisfiable.

INTUITION: would be interesting to run the program on Julia's laptop - if it has a more powerful CPU, *it must solve much faster*.

LLM:

Bottom line: Clingo is CPU-bound, and throwing more cores at it helps only up to a point. For best performance, optimize your encoding and use a modern CPU with strong single-threaded performance (e.g., Intel Core i9 or AMD Ryzen 9).

So, the next good thing to do would be to run this on Julia's laptop and compare speeds - maybe my encoding is not all that bad (it mustn't be), but part of the problem is in the weakness of my CPU.

09.06.25

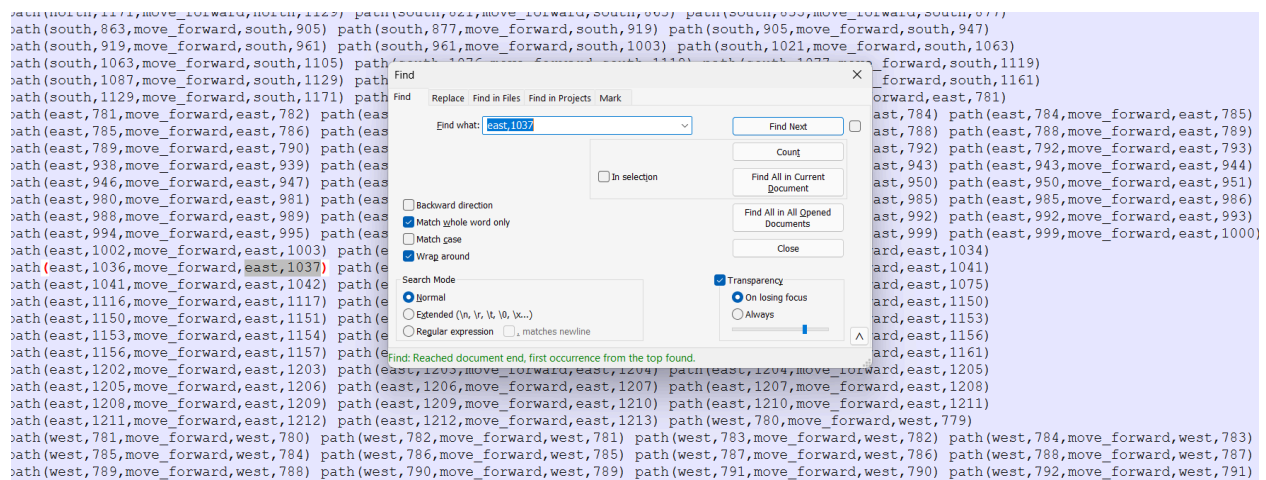
What am I to do now? Well, I want to experiment more with large environments, since with big ones it works impeccably. Just run the big ones and observe. As far as I remember, the first one worked well even when urged to produce a 1000 answers. Let's summarize the situation:

Kep's Test Env	What happens
1st 30 x 30	Solves easily
2nd 30 x 30	Even 1 train is very slow to run (the first train). It can't find even 1 solution. Weird. Let's run the second train separately: better, it's solved in (Solving: 1.34s 1st Model: 1.33s Unsat: 0.00s). The next step is with this one would be to run it on Mac.
42 x 42	Running 1 train: 'Solving' stays a long time... UNSATISFIABLE - how can this be? This must be debugged! One theory is that it is due to a slip in path-transition descriptions. The solving time is acceptable: (Solving: 24.55s 1st Model: 0.00s Unsat: 24.55s) Running the 2nd train: solved instantly: (Solving: 0.07s 1st Model: 0.07s Unsat: 0.00s)

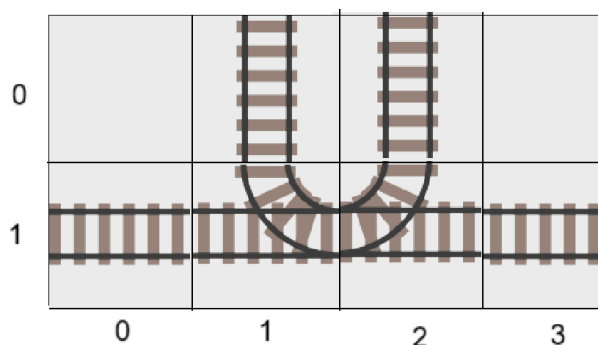
Now, let's debug the 1st train in the biggest environment - why doesn't it go? Let's debug classically through backtracking: opening the image. No, let's visualize it once again. Let's backtrack there to find the bug hides. No, this visualization is messy. I need a new one - with smaller text along x and y axes and without the in-tile text. Yes, it's much cleaner without the in-tile indexes. Let's now also make the axes text smaller. **Done**. Let's now locate the start and the end of the track for the train that falters:

Backtracking: test1: success; test2: success; test3: success; test4: fails => searching for the source of the problem: suspect is track type 35889. Checking: found: it's just not described in the paths *at all*: fixing: **Done**. But let's check the description more carefully to not suspect bugs here later. **Mistake spotted**: direction to the west is not described: **fixed**.

Running the full track for the first train: 'Solving' (intuition: it takes a lot of time because a lot of leeway is given). Still unsatisfiable. Strange. Backtracking again. Debugging: still can't find the bug. I'm sure the bug is stupid, but I can't see it yet. Trying to switch off constraints one by one: **didn't help**. Intuition: something's wrong with the positional converter. Checking: the number of cells is correct: $42 * 42 = 1764$ (1763 in 0 based indexing). For debugging, let's display the grid again with 1d indexes and then check if there is PATH there: FOUND:



The problem was in a tiny bug describing the 'path' normal rule - as a result, 'path' was not created. Fixing: **Done**. Testing: **still fails**. Can't understand why. The last resort: create a small test environment that includes exactly the troubled segments and explore the behaviour:



Unsatisfiable! Why?! Forgot to switch off a temporary constraint! Running again: **FOUND the bug**: the outgoing direction for 1097 was not indicated!

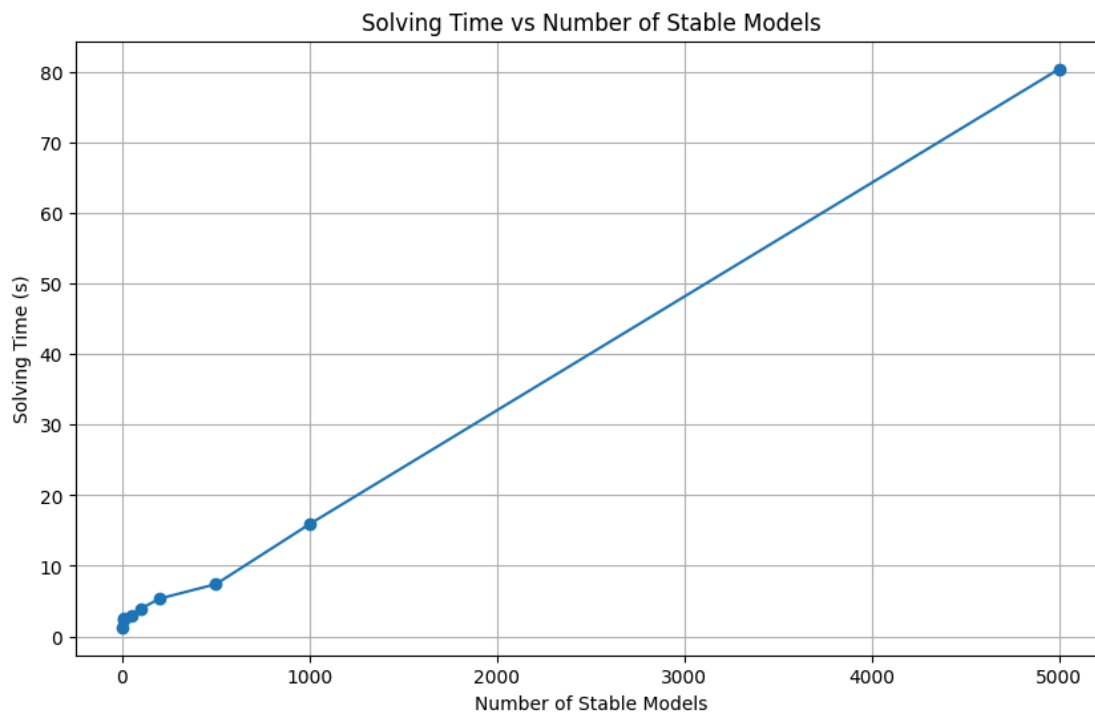
Now, let's jump back to the big test environments: getting bold and running the 42 x 42 instance as a whole: (Solving: 1.28s 1st Model: 1.27s Unsat: 0.00s)

Let's measure different numbers of answer sets and then plot against computation time:

Answer Sets	Time
1	(Solving: 1.28s 1st Model: 1.27s Unsat: 0.00s)
5	(Solving: 2.31s 1st Model: 2.24s Unsat: 0.00s)
10	(Solving: 2.59s 1st Model: 2.48s Unsat: 0.00s)
50	(Solving: 3.01s 1st Model: 2.44s Unsat: 0.00s)
100	(Solving: 3.94s 1st Model: 2.44s Unsat: 0.00s)

200	(Solving: 5.35s 1st Model: 2.47s Unsat: 0.00s)
500	(Solving: 7.40s 1st Model: 2.53s Unsat: 0.00s)
1000	(Solving: 15.92s 1st Model: 2.59s Unsat: 0.00s)
5000	(Solving: 80.46s 1st Model: 2.55s Unsat: 0.00s)

Now let's plot the experiment results:

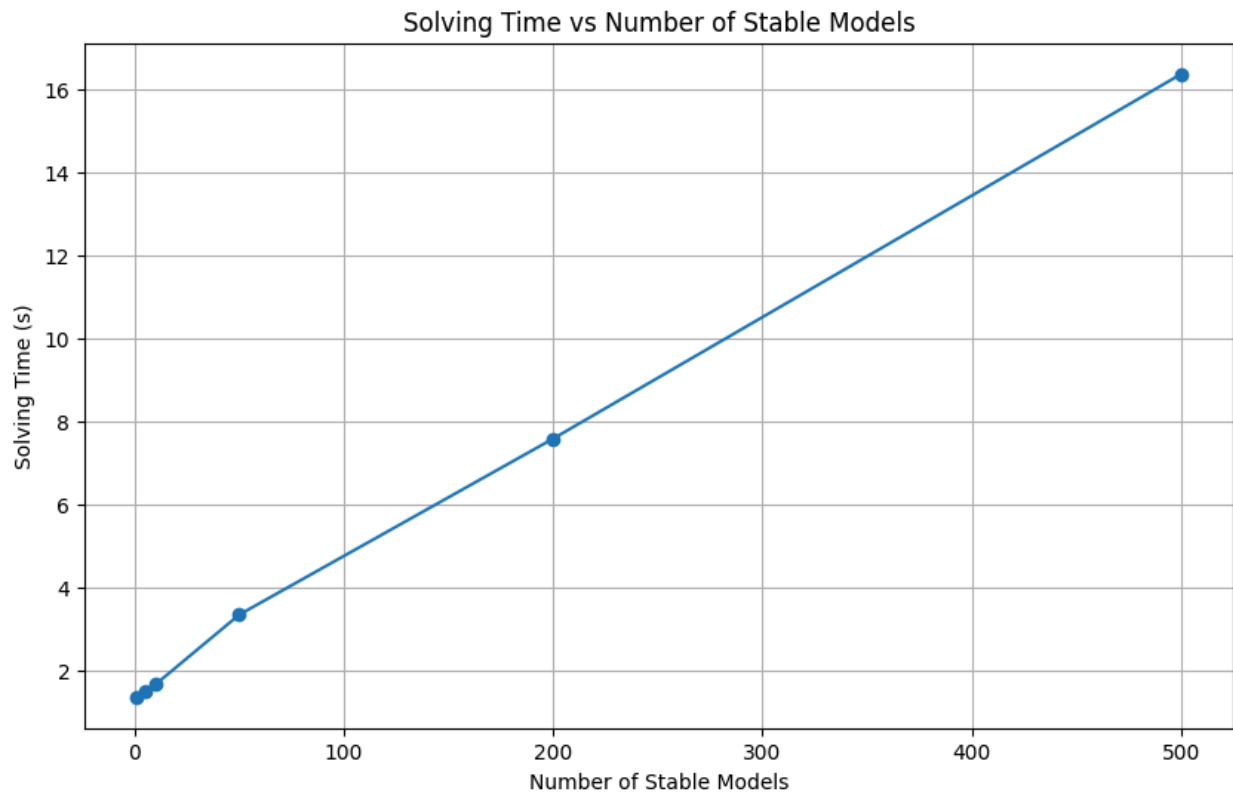


Let's now get back to the second environment - it was the toughest nut for our encoding: running both trains at once. Grounding takes a lot of time. Solving? WOW! (Solving: 1.38s 1st Model: 1.35s Unsat: 0.00s)

Let's order more models?

Answer Sets	Time
1	(Solving: 1.38s 1st Model: 1.35s Unsat: 0.00s)
5	(Solving: 1.53s 1st Model: 1.42s Unsat: 0.00s)
10	(Solving: 1.68s 1st Model: 1.41s Unsat: 0.00s)

50	(Solving: 3.36s 1st Model: 1.40s Unsat: 0.00s)
200	(Solving: 7.60s 1st Model: 1.36s Unsat: 0.00s)
500	(Solving: 16.38s 1st Model: 1.37s Unsat: 0.00s)



CONCLUSION: By now we have arrived at a stable, efficient model that solves all large test instances in a reasonable time. There is still space for optimization (see later), but we can conclude that we've completed the mandatory part of the project.

INTUITION: OPTIMIZATION: a train can only wait before a junction, not after.

OBSERVATION: the third environment is actually SMALL, so it must not cause any performance issues.

INTUITION: would be great to learn to visualize outputs - good for debugging.

- b. Alternative branch to go would be to run a few trains in Kep's environment such that they don't require waiting - just to see the speed.

